

Hessian Spectral Density Estimation: Formalism, Implementation, and Observables

eigengrokking

July 31, 2026

Abstract

This note documents the spectral analysis module (`analysis/eigenthings.py`, `analysis/spectral_observables.py`) used to track the loss Hessian’s eigenvalue distribution over training. It covers, in order: the mathematical formalism (Hessian-vector products, Lanczos tridiagonalization, Stochastic Lanczos Quadrature), how the code implements that formalism, the eight scalar observables extracted per checkpoint and their exact code-level definitions, and how the two governing hyperparameters — Lanczos depth m and probe count k — trade off against estimation accuracy for each observable. The accuracy claims in Section 4 are empirical, taken from `reports/spectral_validation.md`, which validates the estimator against two exact-reference fixtures; see that document for the underlying sweeps.

Contents

1	Mathematical formalism	2
1.1	Setup: the Hessian and its spectral measure	2
1.2	Hessian-vector products via double backpropagation	2
1.3	Lanczos tridiagonalization	2
1.4	Stochastic Lanczos Quadrature	3
1.5	Kernel density estimate for the plotted spectrum	3
2	Implementation: how eigenthings.py applies the formalism	3
2.1	The Eigenthings class	3
2.2	Probe sampling: <code>rademacher_probe</code>	4
2.3	<code>estimate_density</code> : orchestrating one spectral snapshot	4
2.4	Integration into the training loop	5
3	Observables	5
3.1	Pooling probes into one discrete measure	5
3.2	<code>top_eig</code>	5
3.3	<code>bulk_edge</code>	5
3.4	<code>outlier_count</code>	6
3.5	<code>trace</code>	6
3.6	<code>spectral_entropy</code> and <code>effective_rank</code>	6
3.7	<code>negative_mass</code>	7
3.8	<code>conditioning</code>	7
3.9	Error bars: <code>stderr</code> over probes	7

4 Accuracy: how m and k govern each observable	7
4.1 What m and k each control	7
4.2 Per-observable convergence	8
4.3 Cost and production defaults	9

1 Mathematical formalism

1.1 Setup: the Hessian and its spectral measure

Let $\theta \in \mathbb{R}^n$ be the trainable parameters of the model (flattened across all tensors) and $\mathcal{L}(\theta)$ the scalar loss evaluated on a fixed batch. The object of interest is the Hessian

$$H = \nabla_{\theta}^2 \mathcal{L}(\theta) \in \mathbb{R}^{n \times n},$$

a real symmetric matrix with eigendecomposition $H = V \Lambda V^{\top}$, eigenvalues $\lambda_1 \leq \dots \leq \lambda_n$. Since H is *not* a Gauss–Newton or Fisher approximation, it is generically indefinite: negative eigenvalues correspond to directions of negative curvature at θ .

For n in the 10^3 – 10^7 range, forming H explicitly is infeasible. The quantity we actually want is the *empirical spectral measure*

$$\mu_H = \frac{1}{n} \sum_{i=1}^n \delta(\lambda - \lambda_i),$$

which we only ever access indirectly, through two primitives: (i) Hessian-vector products $v \mapsto Hv$, and (ii) an algorithm – Stochastic Lanczos Quadrature (SLQ) – that turns a modest number of HVPs into an estimate of μ_H and of linear functionals of it (trace, tail mass, entropy, $\dots$).

1.2 Hessian-vector products via double backpropagation

Pearlmutter’s trick computes Hv exactly, without materializing H , by differentiating the loss twice. Writing $g(\theta) = \nabla_{\theta} \mathcal{L}(\theta)$,

$$Hv = \nabla_{\theta} \left(g(\theta)^{\top} v \right),$$

because $\nabla_{\theta} (g^{\top} v) = \nabla_{\theta} (v^{\top} \nabla_{\theta} \mathcal{L}) = (\nabla_{\theta}^2 \mathcal{L}) v = Hv$ (v held fixed, not a function of θ). Concretely: one backward pass with `create_graph=True` produces g as a differentiable tensor; a second backward pass through the scalar $g^{\top} v$ yields Hv . This is the *true* Hessian, not a Gauss–Newton/Fisher surrogate — important here since the surrogate is positive semi-definite by construction and would zero out exactly the negative-curvature signal several observables below are built to detect.

1.3 Lanczos tridiagonalization

Given a unit-norm starting vector q_1 , m steps of Lanczos build an orthonormal basis $Q_m = [q_1 \mid \dots \mid q_m]$ of the Krylov subspace $\mathcal{K}_m(H, q_1) = \text{span}\{q_1, Hq_1, \dots, H^{m-1}q_1\}$ in which H restricted to that subspace is tridiagonal:

$$Q_m^{\top} H Q_m = T_m = \begin{pmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \beta_2 & \\ & \ddots & \ddots & \ddots \\ & & \beta_{m-1} & \alpha_m \end{pmatrix}.$$

The basis and the tridiagonal entries are produced together by the three-term recurrence

$$\alpha_i = q_i^\top H q_i, \quad w_i = H q_i - \alpha_i q_i - \beta_{i-1} q_{i-1}, \quad \beta_i = \|w_i\|, \quad q_{i+1} = w_i / \beta_i,$$

with $\beta_0 q_0 \equiv 0$. Each step costs exactly one HVP. In finite precision, orthogonality among the q_i decays after a few dozen steps, so m in the 50–100 range requires *full reorthogonalization* — each new w_i is Gram–Schmidt’d against every previously computed $q_1, \dots, q_i$, not just the last one or two – to keep T_m an accurate reduction of H .

1.4 Stochastic Lanczos Quadrature

Diagonalize the small $m \times m$ matrix $T_m = U \Theta U^\top$ (cheap: $O(m^2)$ – $O(m^3)$, on the CPU). Gauss quadrature theory says that for any sufficiently smooth scalar function f ,

$$q_1^\top f(H) q_1 \approx \sum_{j=1}^m \tau_j^2 f(\theta_j), \quad \tau_j := U_{1j},$$

where the θ_j (eigenvalues of T_m , the *Ritz values*) are the quadrature *nodes* and τ_j^2 (squared first components of T_m ’s eigenvectors) are the quadrature *weights*; $\sum_j \tau_j^2 = 1$ since U is orthogonal. This is exact whenever H restricted to $\mathcal{K}_m(H, q_1)$ captures the relevant spectral content (e.g. exactly, for any $m \geq 1$, when f is affine – see the trace estimator in Section 3.5); more generally it is the degree- $(2m-1)$ moment-matching approximation to $\int f d\mu_{q_1}$ for the measure μ_{q_1} induced by q_1 .

A single probe only sees H through one starting direction. Stochastic Lanczos Quadrature averages over k independent random probes $q_1^{(1)}, \dots, q_1^{(k)}$, each a normalized Rademacher vector (i.i.d. ± 1 entries). Since $\mathbb{E}[vv^\top] = I$ for such v ,

$$\frac{1}{k} \sum_{l=1}^k (q_1^{(l)})^\top f(H) q_1^{(l)} \xrightarrow{k \rightarrow \infty} \frac{1}{n} \text{tr} f(H) = \int f(\lambda) d\mu_H(\lambda).$$

Each probe l contributes a discrete measure $\hat{\mu}^{(l)} = \sum_j (\tau_j^{(l)})^2 \delta(\lambda - \theta_j^{(l)})$; pooling all k probes with weight $1/k$ each gives one atomic measure $\hat{\mu} = \frac{1}{k} \sum_{l=1}^k \hat{\mu}^{(l)}$ that approximates μ_H . Every observable in Section 3 is a functional of this pooled (θ, w) atom set.

1.5 Kernel density estimate for the plotted spectrum

For the plotted density curve (not the scalar observables – see Section 3.1), each atom is replaced by a Gaussian kernel of bandwidth σ :

$$\hat{\phi}(t) = \frac{1}{k} \sum_{l=1}^k \sum_{j=1}^m (\tau_j^{(l)})^2 \mathcal{N}(t; \theta_j^{(l)}, \sigma^2), \quad \mathcal{N}(t; \mu, \sigma^2) = \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{(t - \mu)^2}{2\sigma^2}\right).$$

2 Implementation: how eigenthings.py applies the formalism

2.1 The Eigenthings class

`Eigenthings.__init__` takes the model, a fixed `loss` tensor (already the result of one forward pass on a batch), and the parameter list `params`. It computes the first-order gradient once,

```
self.grads = torch.autograd.grad(loss, params, create_graph=True, allow_
unused=True)
```

This is the $g(\theta)$ of Section 1.2, kept as a differentiable graph node. Because $\mathcal{L}$ and hence g are fixed for the duration of one spectral snapshot, computing g once in `__init__` and reusing it across every `hvp()` call (rather than re-walking the whole backward pass per HVP) is what makes the many HVPs a Lanczos run needs affordable.

`hvp(q)` implements Section 1.2 on a flattened probe vector $q \in \mathbb{R}^n$: `unpack_vec` reshapes q into one tensor per parameter (matching each p 's shape), $g^\top v = \sum_p (g_p \cdot v_p)$.`sum()` is accumulated, and a second `torch.autograd.grad` call differentiates that scalar back through `self.params` to produce Hv (again per-parameter, then re-flattened by `pack_list`). Missing gradients (`allow_unused=True`, for parameters the loss doesn't depend on) are treated as zero.

`lanczos(v0)` runs the recurrence of Section 1.3 for `self.m` steps: `q/q_prev` track q_i, q_{i-1} ; `alpha/beta` accumulate α_i, β_i ; `Q` keeps every basis vector generated so far. When `self.orthogonalize` is set (the production default), each new w is explicitly reduced against *every* vector in `Q`, not just the most recent one — the full reorthogonalization called out in Section 1.3. The loop terminates early if $\beta_i < 10^{-12}$ (the Krylov subspace has become invariant — no new direction to explore), so the realized depth $k = \text{len}(\text{alpha})$ can be less than `self.m`. The tridiagonal T_m is assembled explicitly from `alpha/beta` and returned.

`lanczos_quadrature(v0)` calls `lanczos`, moves the (small) T_m to CPU for `torch.linalg.eigh`, and reads off nodes $\theta_j = \text{eigvals}$ and weights $\tau_j^2 = \text{eigvecs}[0, :]**2$ exactly as in Section 1.4.

`density_from_lanczos(nodes, weights, t_grid, sigma)` evaluates the KDE sum of Section 1 on an arbitrary grid, skipping atoms whose weight is negligible ($|\tau_j^2| \leq 10^{-14}$) for efficiency.

2.2 Probe sampling: rademacher_probe

`rademacher_probe` draws ± 1 entries via a CPU `torch.Generator` and only moves the result to the target device afterwards, because CUDA/MPS generators are not bitwise-interchangeable with the CPU one — this keeps the probe sequence identical regardless of accelerator, which matters for reproducibility across runs/hardware.

2.3 estimate_density: orchestrating one spectral snapshot

`estimate_density(model, m, k, sigma_frac, loss, probe_seed, ...)` is the entry point used once per logged training step. It runs in two phases:

1. **Cheap top-eigenvalue pre-pass.** A single Rademacher probe (seeded from `probe_seed`, its own generator so it does not disturb the sequence used below) is run through `lanczos_quadrature` with a shallow depth `top_eig_probe_m` (default 20 — see Section 4, `top_eig` converges by $m \geq 20$). The largest node gives a cheap estimate of $|\lambda_{\max}|$. This exists solely to set the absolute KDE bandwidth: $\sigma = \max(\text{sigma_frac} \cdot |\lambda_{\max}|, \text{eps})$, a *fraction* of $\lambda_{\max}$ rather than a fixed absolute value, since $\lambda_{\max}$ itself can move by orders of magnitude over a training run and a fixed σ would make density plots incomparable across checkpoints.
2. **Main k -probe, m -step pass.** The same `Eigenthings` instance (reusing its once-computed `grads`) is reconfigured to depth `m`, and a fresh generator reseeded from the *same* `probe_seed` draws k Rademacher probes. Calling `estimate_density` repeatedly with the same `probe_seed` (e.g. once per checkpoint of a training run) therefore draws bit-identical probes every time, isolating parameter-driven spectral change across checkpoints from probe-sampling noise.

The plotting grid `t_grid` spans $[\min(\text{nodes}) - 6\sigma, \max(\text{nodes}) + 6\sigma]$ at step $\sigma/5$, clamped to $[200, 50000]$ points so that a very wide node range (e.g. right after the top-eig pre-pass makes σ very small) cannot silently blow the grid past a usable size. `SpectralEstimate` bundles both outputs: the KDE-smoothed `density/t_grid` (plotting only) and the raw, un-smoothed per-probe `probe_nodes/probe_weights` (everything in Section 3).

2.4 Integration into the training loop

`training/loop.py` calls `estimate_density` on a schedule (`SpectralSchedule`, geometric-then-dense around the train/test-accuracy-gap transition), always against the *same* fixed batch (`spectral_batch_size`, default 512, sampled once before training starts) so that, per snapshot, only θ differs and neither the data nor the probes are confounds. The raw nodes/weights returned by `estimate_density` are passed straight into `compute_spectral_observables_with_stderr` (Section 3), and both the point estimates and their stderrs are logged into the training history per checkpoint.

3 Observables

All eight observables below are implemented in `analysis/spectral_observables.py`, `compute_spectral_observables`, and are computed from the *raw* pooled Ritz atoms, not the KDE-smoothed density — the smoothing bandwidth would otherwise bias tail-sensitive quantities such as `top_eig` and outlier detection.

3.1 Pooling probes into one discrete measure

`_pool_probes` concatenates the k probes' $(\theta^{(l)}, \tau^{(l)2})$ pairs into one atom set, dividing each probe's weights by k so the pooled weights sum to 1 overall (each probe contributes total mass $1/k$, matching the SLQ average of Section 1.4):

$$\{(\theta_j, w_j)\}_{j=1}^N, \quad N = \sum_{l=1}^k (\text{realized depth of probe } l), \quad w_j = \tau_j^{(l)2}/k.$$

Below, $\lambda_{\max} := \theta_N$ after sorting ascending (the largest pooled node), and n denotes `n_params`, the true parameter count — *not* N , which scales with m and k and would make several observables below drift with Lanczos depth/probe count rather than track the actual spectrum.

3.2 top_eig

Mathematically, the estimate of $\lambda_{\max}(H)$. Code: the largest pooled node, `nodes[-1]` after sorting. Precise because it is read off the raw atoms rather than the KDE curve, which would blur/underestimate the tail.

3.3 bulk_edge

A robust threshold separating the low-curvature “bulk” of the spectrum from outlier eigenvalues, defined as a weighted median plus a multiple of the weighted median absolute deviation (MAD):

$$\text{bulk_edge} = \text{median}_w(\theta) + c \cdot 1.4826 \cdot \text{MAD}_w(\theta), \quad c = \text{bulk_mad_multiplier} = 5,$$

where median_w and $\text{MAD}_w = \text{median}_w(|\theta - \text{median}_w(\theta)|)$ are computed against the *weighted* CDF of the pooled atoms (`_weighted_quantile`, linear interpolation of q against $\text{cdf}_j = (\sum_{i \leq j} w_i -$

$\frac{1}{2}w_j)/\sum_i w_i$, i.e. each atom’s cumulative-probability coordinate sits at its own mass’s midpoint), not a plain `numpy.median` over the raw node values. The 1.4826 factor rescales MAD to be consistent with a Gaussian standard deviation. This weighting is not cosmetic: Section 4 documents that the unweighted version was found to be biased 30–50% on a spectrum with a realistic bulk/outlier imbalance, because Lanczos node placement is moment-matching, not density-matching – nodes cluster near spectral edges regardless of how much true probability mass sits there, so treating every node as equally likely (unweighted) systematically overweights the edges.

3.4 outlier_count

The number of *distinct* eigenvalues sitting above `bulk_edge`, as opposed to the number of Ritz values above it (Lanczos rediscovers the same true outlier eigenvalue as several near-identical Ritz values across different probes/depths). Code: take pooled nodes with $\theta_j > \text{bulk_edge}$, sort, and count runs separated by gaps larger than `resolution` = $\max(\text{resolution_frac} \cdot |\lambda_{\max}|, \text{eps})$ (default `resolution\_frac` = 0.05) – i.e. the number of gaps exceeding `resolution`, plus one.

3.5 trace

The Hutchinson-via-Lanczos trace estimator,

$$\widehat{\text{tr}}(H) = n \sum_{j=1}^N w_j \theta_j.$$

This follows directly from Section 1.4 with $f(\lambda) = \lambda$: $\mathbb{E}_v[v^\top H v] = \text{tr}(H)$ for Rademacher v , and $q_1^\top H q_1 = \sum_j \tau_j^2 \theta_j$ is exact for *any* $m \geq 1$ — it is the first-moment identity of Gauss quadrature (in fact $q_1^\top H q_1 = \alpha_1$, the very first Lanczos coefficient). This is why, uniquely among the eight fields, `trace`’s accuracy is governed entirely by k (Monte Carlo/Hutchinson variance, $O(k^{-1/2})$) and is essentially insensitive to m .

3.6 spectral_entropy and effective_rank

The Shannon entropy of a *magnitude-reweighted* spectral measure. Model each of the n exact eigenvalues λ_i as carrying uniform prior mass $1/n$, reweighted by $|\lambda_i|$ (so directions with negligible curvature, positive or negative, don’t count toward “effective rank”):

$$p_i = \frac{|\lambda_i|}{nZ}, \quad Z := \frac{1}{n} \sum_{i=1}^n |\lambda_i| \approx \sum_{j=1}^N w_j |\theta_j| \quad (\text{pooled SLQ estimate of } \frac{1}{n} \sum_i |\lambda_i|).$$

Then

$$S = - \sum_{i=1}^n p_i \log p_i = \log(nZ) - \frac{1}{nZ} \sum_{i=1}^n |\lambda_i| \log |\lambda_i| \approx \log(nZ) - \frac{1}{Z} \sum_{j=1}^N w_j |\theta_j| \log |\theta_j|,$$

using the same pooled-SLQ substitution for $\frac{1}{n} \sum_i (\cdot)$ in the second term. This is exactly `spectral_entropy` in the code (`z = sum(weighted_abs)`, `cross_term = sum(weighted_abs * log_abs)`, `spectral_entropy = log(n_params*z) - cross_term/z`), computed in closed form off the raw atoms with no histogram binning, so there is no artificial cap on the result. `effective_rank` = e^S is the “perplexity” of p : the number of eigen-directions (out of n) carrying non-negligible $|\text{curvature}|$, regardless of sign. Because n (the true parameter count) rather than N (the pooled Ritz-node count, which scales with mk) appears in the $\log(nZ)$ term, S and e^S are comparable across different m, k settings rather than drifting with them.

3.7 negative_mass

The probability mass at eigenvalues meaningfully below zero, *margin-thresholded* rather than a raw $\lambda < 0$ count:

$$\text{negative_mass} = N_\epsilon = \sum_{j: \theta_j < -\epsilon} w_j, \quad \epsilon = \text{margin_c} \cdot |\lambda_{\max}|, \quad \text{margin_c} = 0.05.$$

The margin exists because Lanczos produces spurious small-magnitude negative Ritz values immediately around a true eigenvalue near zero; a raw < 0 threshold counts that numerical noise as curvature evidence (see Section 4 for the measured effect). `margin_c` shares its default with `resolution_frac` (both are “a small fraction of $|\lambda_{\max}|$ ” length scales) but is an independently tunable knob: one sets a spectral resolution for clustering, the other a decision boundary for “genuinely negative.”

3.8 conditioning

A cheap proxy for how far the extreme eigenvalue sits above the bulk,

$$\text{conditioning} = \frac{\text{top_eig}}{\text{bulk_edge}},$$

undefined (NaN) if $|\text{bulk_edge}| \leq \text{eps}$. Chosen over a fixed absolute threshold or a Marchenko–Pastur-style edge (which needs an aspect ratio not well-defined for a Hessian).

3.9 Error bars: stderr over probes

`compute_spectral_observables_with_stderr` returns (point, stderr): `point` pools all k probes (as above, the least-biased estimate); `stderr` recomputes every observable k times, once per single-probe atom set, and reports the standard error of the mean across those k values, $\text{sd}(\{x^{(l)}\}_{l=1}^k)/\sqrt{k}$ with $\text{ddof} = 1$. This costs no extra HVPs (it reuses the already-computed per-probe atoms) and is a pragmatic uncertainty proxy rather than the literal standard error of every point estimate – e.g. `top_eig` is a pooled *max*, not a mean of per-probe maxes, so the point estimate is not the mean of the values whose spread `stderr` reports.

4 Accuracy: how m and k govern each observable

4.1 What m and k each control

m (Lanczos steps per probe, `spectral_m`) sets the depth of the Krylov subspace built per probe, i.e. *algorithmic* convergence: how well the small T_m ’s Ritz values approximate the true eigenvalues H has along that probe’s Krylov direction. k (probe count, `spectral_k`) sets how many independent Rademacher directions are averaged, i.e. *Monte Carlo* convergence of the Hutchinson-style average in Section 1.4, governed by the CLT ($O(k^{-1/2})$ error). These are different axes, and — this is the central empirical finding of `reports/spectral_validation.md` — different observables are bottlenecked by different axes, so no single (m, k) is simultaneously “cheapest” for every field. Total cost per spectral snapshot is $\approx m \cdot k$ HVPs (one HVP per Lanczos step, per probe), and this snapshot recurs at every logged step of a training run, so (m, k) is the dominant lever on the recurring cost of `run_spectral=True`.

4.2 Per-observable convergence

The table below summarizes the sweeps in `reports/spectral_validation.md` against two exact-reference fixtures: a ~ 1026 -parameter random-init MLP with a materialized Hessian (`tests/reference_hessian.py`), and an engineered quadratic-form fixture with the same parameter count but a production-like spectral dynamic range (exact `top_eig`= 150, `bulk_edge`= 1.109, `conditioning`= 135.3, i.e. $\sim 31\times$ wider than the first fixture), built specifically to exercise `bulk_edge/conditioning`, which the near-flat first fixture could not. *The qualitative governing axis (m vs. k) is a property of the SLQ algorithm and should generalize; the specific numeric thresholds are shaped by each fixture’s spectral gap and should not be copy-pasted onto a production run without their own check.*

Observable	Governing axis	Min. m	Min. k
<code>trace</code>	k (m irrelevant)	1	≥ 100 (≥ 300 tighter)
<code>top_eig</code>	m	≥ 20	1
<code>negative_mass</code> (margin-thresh.)	m	≥ 30 (50 for full margin)	≥ 30
<code>spectral_entropy</code>	m	≥ 20 (30 w/ margin)	≥ 30
<code>effective_rank</code>	m	≥ 20 (30 w/ margin)	≥ 30
density (plotted curve)	m , then k	≥ 20 (30 w/ margin)	≥ 100
<code>bulk_edge</code> (post-fix)	m	≥ 75 (100 for $<0.2\%$)	not swept
<code>conditioning</code> (post-fix)	m	≥ 75 (100 for $<0.2\%$)	not swept

Notable points behind the table:

- `trace` converges purely in k because, as shown in Section 3.5, the underlying quadrature identity is exact in m for the first moment; all error is Hutchinson/Monte-Carlo variance. Measured RMS relative error ($R = 20$ independent repeats) was 21.0% at $k=10$, 4.6% at $k=100$, 2.2% at $k=1000$ — $\sqrt{k}$ convergence confirmed at a fitted log-log slope of -0.511 against a -0.5 theoretical rate. Reaching $\sim 1\%$ relative error would need k in the low thousands.
- `negative_mass` was originally defined with a raw $\theta < 0$ threshold and needed $m \geq 200$ to converge (non-monotonically in m) — traced to 193 of 1026 exact eigenvalues on the reference fixture sitting within 5% of $|\lambda_{\max}|$ of zero, a band where Lanczos needs substantial depth just to resolve the *sign* correctly. Switching to the margin-thresholded definition of Section 3 (excluding that band by construction) dropped the requirement an order of magnitude, to $m \geq 30$ –50.
- `bulk_edge/conditioning` were the deepest m -governed fields once correctly estimated, but their original implementation (plain, *unweighted* `numpy.median`/MAD over the pooled Ritz nodes) was found to be 30–50% biased *at every m tested, including $m=300$* , with negligible probe-to-probe spread — the signature of a biased estimator, not under-convergence, since raising m further would not have fixed it. This was an estimator bug (Section 3’s weighted-median fix), not a parameter-tuning problem; only after the fix does the $m \geq 75$ –100 threshold in the table reflect genuine Lanczos-depth convergence.
- `spectral_entropy/effective_rank` converge an order of magnitude faster in m than `negative_mass` despite reading the same pooled $|\lambda|$ distribution, because the entropy’s closed form (Section 3) only needs Ritz mass to land in roughly the right coarse bin, whereas `negative_mass` needs the exact sign right at a zero crossing — a much finer distinction that coarse convergence can’t average over.

4.3 Cost and production defaults

Current production defaults, `configs/base.py`:

```
spectral_m = 50,    spectral_k = 100,
```

i.e. $\approx 5,000$ HVPs per logged spectral snapshot. Cross-referencing against the table above: $k=100$ is a deliberate compromise for `trace` ($\sim 5\%$ relative error, not the $\sim 1\%$ original target, since that would need k in the low thousands and k multiplies snapshot cost directly for a quantity logged repeatedly through training, not once); $m=50$ comfortably clears the $m \geq 20\text{--}30$ threshold shared by `top_eig`, `negative_mass`, `spectral_entropy`, and `effective_rank`, but sits *below* the $m \geq 75\text{--}100$ threshold the validation doc establishes for `bulk_edge/conditioning` to reach sub-1.5%/sub-0.2% relative error post-fix. (The inline comment on `spectral_m` in `configs/base.py` cites “ $m=50$, $k=300$ ” as clearing every m -bound field, but the checked-in default sets `spectral_k=100`, not 300, and `spectral_validation.md`’s own summary table puts $m \geq 75$ — not 50 — as the requirement for `bulk_edge/conditioning`; this is a live discrepancy between the code comment and both the configured value and the validation doc’s recommendation, worth resolving directly rather than assuming either side is authoritative.) `bulk_edge/conditioning` have also not been swept over k at all, so their behavior below the $\sim 0.2\%$ level with non-default k is unverified.

Because cost scales as $m \cdot k$, and k is set by `trace` while m is set (once `bulk_edge/conditioning` are the binding constraint) independently of k , the two knobs should generally be tuned separately per analysis rather than raised together: if only `top_eig` or the entropy family is needed, $m \approx 20\text{--}30$ with k as low as 1 is sufficient and far cheaper than the shared production default; if `trace` accuracy tighter than $\sim 5\%$ is needed for a specific run, that is a reason to raise `spectral_k` for that run rather than the global default, given the direct cost multiplier on every logged checkpoint.